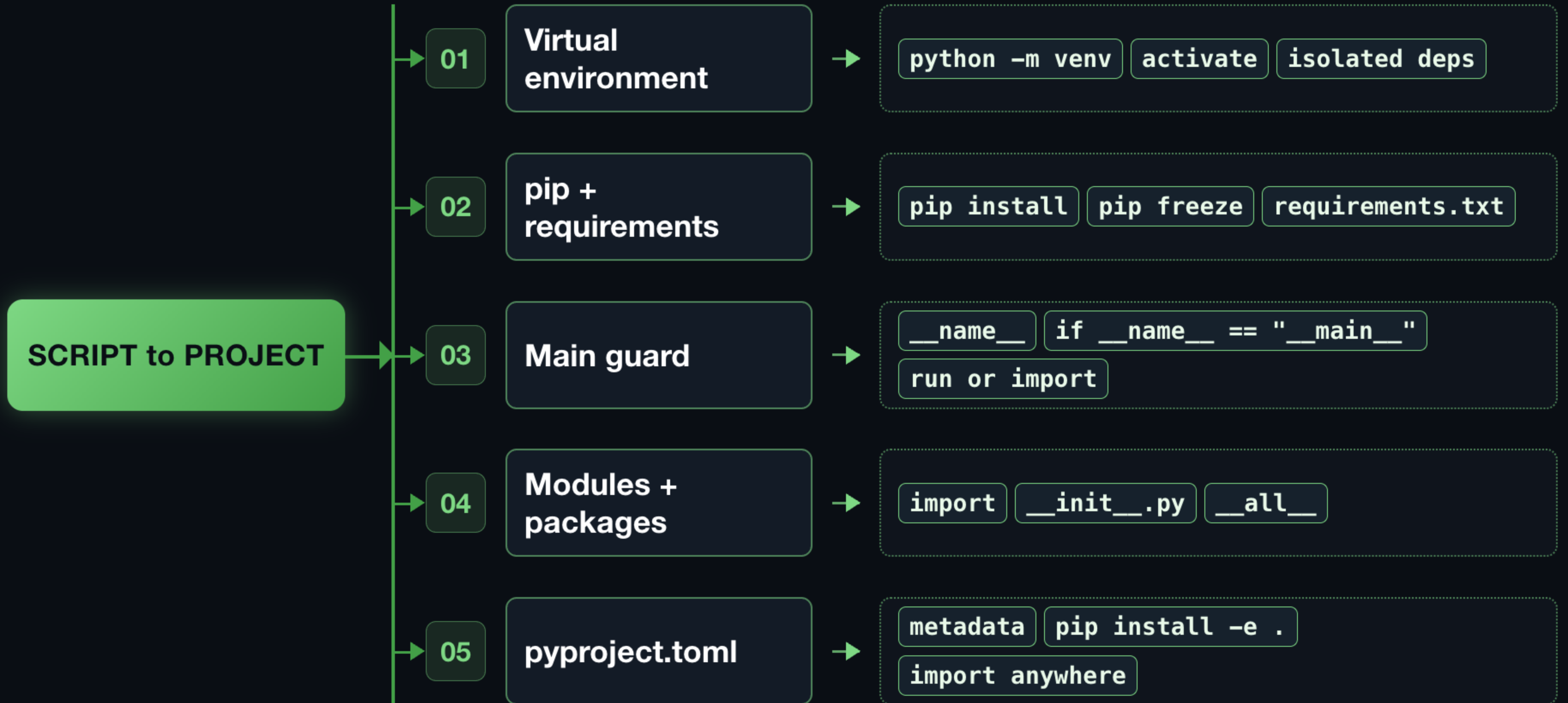


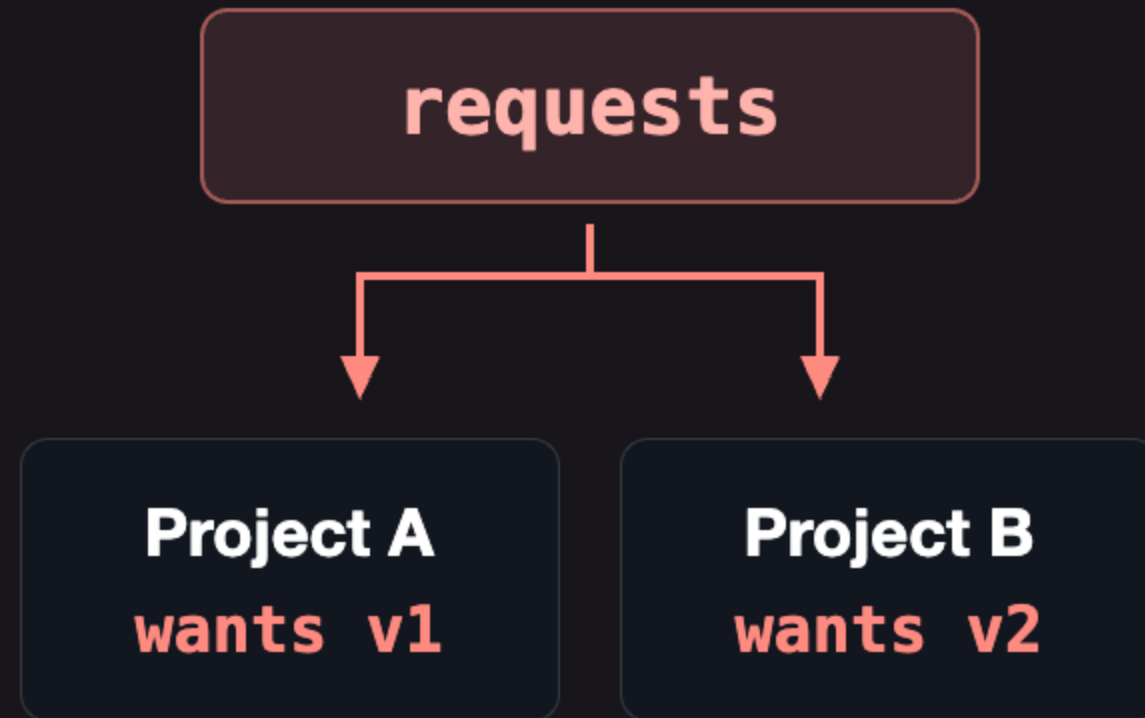
Section 12: Virtual Environments and Packaging **Concept Map**

The path through Section 12: the habits that turn loose scripts into an isolated, installable project.



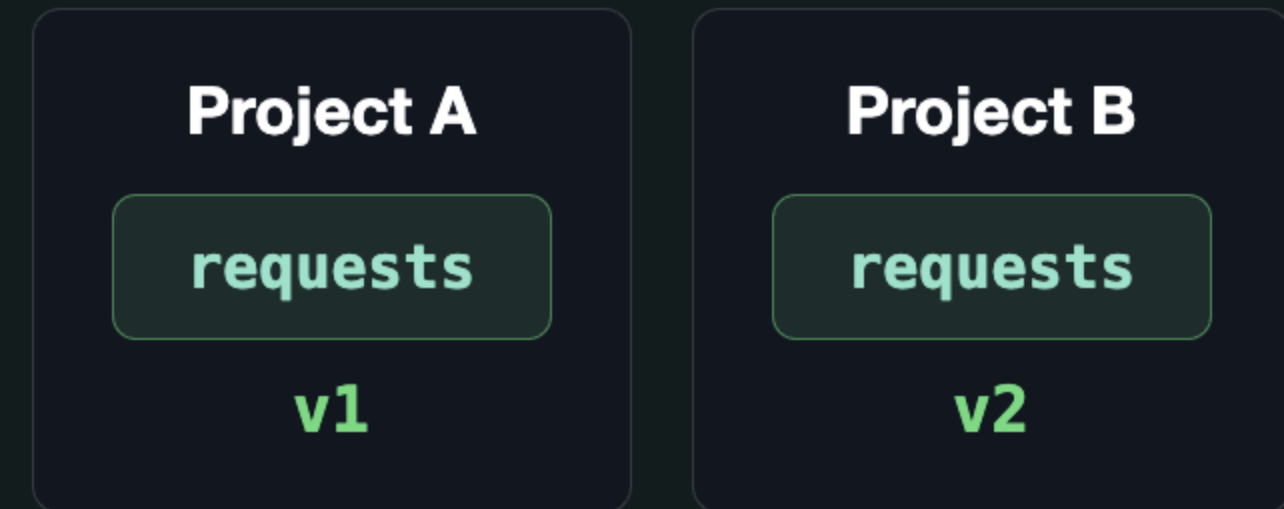
Why each project needs its own **venv** (virtual environment)

One shared Python



Only one can win

One Python each



Both work

What a **venv** actually is

A folder inside your project

my_project/

.venv/

its own python and packages

app.py

number_utils/

That is all it is. Delete the folder and it is gone.

What activating changes

source .venv/bin/activate

points your terminal at that folder

before

python



system Python

after

python



.venv/

So **pip install** lands inside the project, not on your whole machine.

Virtual Environments: what you will do in this step

Build one project folder that carries its own private Python, prove to yourself that it changed, then tidy up after. Every line below is typed in the **terminal**, not in a **.py** file.

#	WHAT YOU DO	COMMAND
1	Create the project and its venv <code>venv</code> A folder for the project, and a private Python inside it.	<code>python3 -m venv .venv</code>
2	Check which Python you are on Look BEFORE you activate, so you have something to compare.	<code>which python3</code>
3	Activate it <code>activate</code> Your prompt then carries the name in front of it.	<code>source .venv/bin/activate</code>
4	See what changed The same question now answers with a path inside your project.	<code>which python</code>
5	Look inside a brand new venv Nearly empty, which is the win. deactivate steps back out.	<code>pip list</code>
6	The name is your choice .venv is only a convention, not a rule.	<code>python3 -m venv my_environment</code>
7	A second venv, then clean up One folder can hold two. Leave one, enter the other, delete the spare.	<code>rm -rf .venv2</code>

Installing Packages: **what you will do in this step**

Add a package someone else wrote, record exactly what you installed, then DELETE your environment and rebuild it from that record. Every line is typed in the **terminal**.

#	WHAT YOU DO	COMMAND
1	Fresh project, activate its venv pip installs into whichever Python is active. It will not warn you.	<code>mkdir step02_my_project</code>
2	Install a third-party package <code>requests</code> The first command in the course that needs the internet.	<code>pip install requests</code>
3	See what you got You asked for one package and five arrived. The extras are its dependencies.	<code>pip list</code>
4	Record what is installed Every package pinned to its exact version, written to a file.	<code>pip freeze > requirements.txt</code>
5	Rebuild in a fresh environment The proof: an empty venv becomes the same five packages.	<code>pip install -r requirements.txt</code>
6	Clean up Delete the spare and go back to the project venv.	<code>rm -rf .venv2</code>

The Main Guard: what you will do in this step

See what goes wrong when one file imports another, then fix it with `if __name__ == "__main__":`. It gets a small demo project of its own, so nothing else is in the way.

#	WHAT YOU DO	SAMPLE CODE
1	Meet Python's special names A fresh demo folder and venv, then print what Python sets.	<code>print(__name__)</code>
2	Write app.py, without the guard Prints its <code>__name__</code> , then a function and a bare <code>main()</code> .	<code>main()</code>
3	Write runner.py A second file that only wants one function.	<code>from app import double</code>
4	Run it and watch it break <code>main()</code> runs, even though <code>runner.py</code> never asked for it.	<code>python runner.py</code>
5	Add the guard Change only the last line of <code>app.py</code> .	<code>if __name__ == "__main__":</code>
6	Run it both ways Imported, <code>main()</code> stays quiet. Run directly, it still runs.	<code>python app.py</code>

Your Own Package: **what you will do in this step**

Put your own functions in their own file, group them into a package, and import them from another file. The package files you write in **VS Code**.

#	WHAT YOU DO	SAMPLE CODE
1	Fresh project and its venv A new folder beside the last one, with its own venv.	<code>mkdir step04_my_project</code>
2	Make the package and write the module A folder, an empty <code>__init__.py</code> , and the functions.	<code>def double(n):</code>
3	Create app.py and watch it break ImportError: the front door is still empty.	<code>from number_utils import double</code>
4	Open the front door One import line in <code>__init__.py</code> hands the functions out.	<code>from number_utils.calculations import double</code>
5	Run it again Same app.py, now it works.	<code>python app.py</code>
6	Import everything with * Grabs every name the package hands out.	<code>from number_utils import *</code>
7	Limit * with __all__ Named imports are not affected.	<code>__all__ = ["double"]</code>

Module vs Package: one file, and a folder of files

The mental picture

PACKAGE `number_utils/`

a folder that organizes related modules

`__init__.py` the front door: what the package hands out

MODULE `calculations.py`

`double()`

`percentage()`

MODULE `another_module.py`

a package can hold as many related modules as you need

PACKAGE

holds

MODULES

hold

FUNCTIONS

In one line each

MODULE one reusable `.py` file

it holds the functions: `calculations.py`

PACKAGE a folder of related modules

it groups the modules: `number_utils/`

FUNCTION the code you actually call

it lives inside a module: `double()`

Two ways to get `double()`

`from number_utils.calculations import double`
reach into the module

`from number_utils import double`
through the front door, once `__init__.py` hands it out

A module is one reusable `.py` file. A **package** is a folder that organizes related modules.

Environments: what the **-e** in **pip install -e .** does

It does not copy your code anywhere

pyproject.toml describes your project, then **-e** (editable) drops a **pointer** into the venv. Your package becomes importable from any folder, and it still runs from **your own source files**.

FROM ANOTHER FOLDER, BEFORE
from number_utils import double
ModuleNotFoundError:
No module named 'number_utils'

pip install -e .

FROM ANOTHER FOLDER, NOW IT WORKS
from number_utils import double
print(double(25))
50

What actually landed in the venv

pip wrote a small pointer file, not your code:
site-packages/__editable__.number_utils-0.1.0.pth

and your package still runs from where you wrote it:
number_utils.__file__
/step05_my_project/number_utils/__init__.py

So your edits are live, no reinstall

edit **number_utils/calculations.py**
(double: **n * 2** becomes **n * 3**), run again:
75
pip was never run a second time

Drop the **-e** and pip copies your code into the venv instead, so every edit needs a reinstall before it shows up. **Use -e while you are building the package.**

Packaging Your Project: **what you will do in this step**

Make your own package importable from ANY folder, the way **requests** is, by describing the project in a **pyproject.toml** and installing it into the venv.

#	WHAT YOU DO	SAMPLE CODE
1	Start from Step-04's code A fresh folder and venv, with the package copied in.	<code>mkdir step05_my_project</code>
2	Watch it break Step out of the folder and the import fails.	<code>python -c "from number_utils import double"</code>
3	Describe your project <code>pyproject.toml</code> Its name, its version, and how to build it, in one file.	<code>name = "number_utils"</code>
4	Install it into the venv The dot means the project in the folder you are standing in.	<code>pip install -e .</code>
5	Use it from a different project A brand new folder with one file imports your package.	<code>python report.py</code>
6	Change the code, run again, install nothing Edit the module and the other project changes at once.	<code>return n * 3</code>
7	See what pip actually did A pointer file, not a copy. Your own source folder still runs.	<code>pip show number_utils</code>